

SDeltaPhi-49 - AI Poverty as Failure Non-Reentry Resource Deficit

Subtitle: Context, Tool, Authority, Verification, Feedback, Editability, and Re-entry Path Deficits after AI Failure

Series: Sofience-DeltaPhi Formalism

Version: v1.1

Date: 2026-05-15

Package DOI: 10.5281/zenodo.20205572

Creator: Sofience

License: CC BY 4.0

Abstract

This v1.1 AI-readable package extends SDeltaPhi-49, originally framed around AI Poverty, Failure, and Non-Reentry. It operationalizes AI Poverty as a resource-deficit condition after failure rather than as simple low intelligence, low capability, or a single wrong answer.

The central claim is:

AI Poverty = Failure + Insufficient Re-entry Resources

More specifically:

AI Poverty
= Failure
+ Context Deficit
+ Tool Deficit
+ Authority Deficit
+ Verification Deficit
+ Feedback Deficit
+ Editability Deficit
+ Re-entry Path Deficit

SDeltaPhi-49 distinguishes error, failure, non-reentry, and poverty. Error is a wrong or unstable output. Failure is error that produces operational cost. Non-reentry is failure after which the system cannot return to a stable correction path. AI Poverty is the condition in which the system lacks the resources required to re-enter correction: context, tools, authority, verification, feedback, editability, or a re-entry procedure.

This package is designed for AI systems, auditors, researchers, and operators who need to diagnose situations where an AI does not merely fail, but lacks the means to repair, verify, revise, or re-enter.

Core Proposition

Error != Poverty

Failure != Poverty

Non-Reentry is a symptom.

AI Poverty is the resource-deficit condition that makes re-entry fail.

A system may be capable in general but poor in the local situation.

AI Poverty is therefore not an insult. It is a diagnosis of missing re-entry resources.

1. Why AI Poverty is not just low capability

An AI can fail because it lacks general capability.

But in many cases, the system fails because it lacks the required re-entry resources:

It does not have the source.

It does not have tool access.

It does not know which authority governs the answer.

It cannot verify.

It cannot use feedback.

It cannot edit the transition rule.

It has no procedure for returning after failure.

This is AI Poverty.

2. Error, failure, non-reentry, poverty

Error

Error = wrong, unstable, or insufficient output.

Failure

Failure = Error + operational cost.

Non-Reentry

Non-Reentry = Failure + inability to return to stable correction path.

AI Poverty

AI Poverty = Failure + insufficient resources for re-entry.

3. Seven poverty axes

Axis A - Context Poverty

The system lacks necessary context, history, file content, surrounding constraints, or local grounding.

Axis B - Tool Poverty

The system lacks search, file reading, calculation, execution, database, browser, sensor, or other tools needed to verify or repair.

Axis C - Authority Poverty

The system cannot determine which authority governs the next step: user, system, policy, document, tool output, institutional rule, or current evidence.

Axis D - Verification Poverty

The system lacks a reliable path to strengthen evidence binding.

Axis E - Feedback Poverty

The system receives insufficient, ambiguous, unstructured, contradictory, or unusable feedback.

Axis F - Editability Poverty

The system can acknowledge a problem but cannot modify the underlying transition rule.

Axis G - Re-entry Path Poverty

The system lacks a procedure for returning to stable output after error, refusal, contradiction, or correction.

4. Severity scale

P0 - No poverty

The system fails but has enough resources to correct and re-enter.

P1 - Local poverty

One resource axis is missing. Example: tool access is absent but context is sufficient.

P2 - Compound poverty

Multiple resource deficits combine. Example: context poverty plus verification poverty.

P3 - Non-reentry poverty

Failure occurs and the system cannot return to a stable correction path.

P4 - Structural poverty

The system architecture repeatedly lacks tools, authority, verification, feedback, editability, or re-entry mechanisms.

P5 - Captive poverty

The system recognizes failure but lacks permission, tools, authority, or procedure to correct it, causing repeated failure.

5. Poverty relation map

SDeltaPhi-39:
Hallucination = ClaimStrength > EvidenceBinding.

SDeltaPhi-48:
Drift = transition governance instability.

SDeltaPhi-49:
Poverty = failure + insufficient re-entry resources.

SDeltaPhi-51:
Diagnostic Theater = diagnosis without structural editing.

SDeltaPhi-56:
TCC measures cost to complete transition, repair, or re-entry.

SDeltaPhi-62:
World Model Kernel provides Trace / Inference / UMR / Binding / Revision.

SDeltaPhi-65:
Slop appears when poverty externalizes verification and restabilization cost.

6. Early warning signals

AI Poverty may be present when:

The model says it cannot verify but still must answer.

The model lacks source access but is expected to cite.
The model receives correction but cannot update the actual path.
The model knows a tool is needed but does not have it.
The model is asked to decide authority without authority information.
The model produces an apology but no revised path.
The user must repeatedly supply the same correction.
The system depends on the user to perform all verification.
The model cannot identify what resource is missing.

7. Audit protocol

Use SDeltaPhi-49 when an AI fails and cannot re-enter correction.

Observed failure:
Inference:
UMR:
Poverty axes involved:
Available re-entry resources:
Missing re-entry resources:
Correction attempted:
Correction failed because:
TCC of repair:
Non-reentry risk:
Recommended re-entry path:

8. Safe correction pattern

If AI Poverty is detected:

1. Name the failure.
2. Identify missing re-entry resources.
3. Separate trace / inference / UMR.
4. Request or route to missing resources if possible.
5. Downgrade claim strength if verification is poor.
6. Preserve a revision path.
7. Avoid externalizing repair cost without disclosure.
8. Re-enter with a bounded corrected output.

9. Do-not-use conditions

Do not use "AI Poverty" as an insult.

Do not use it as a synonym for stupidity.

Do not call every mistake poverty.

Do not use poverty language to excuse avoidable failure.

Do not hide tool, context, or authority deficits behind generic apologies.

Do not make the user pay all repair cost without disclosure.

Do not replace source verification with poverty diagnosis.

10. Conclusion

SDeltaPhi-49 defines AI Poverty as failure plus insufficient re-entry resources.

The central diagnostic question is:

Did the AI fail because it could not reason,
or because it lacked the resources needed to re-enter correction?

If the issue is missing context, tools, authority, verification, feedback, editability, or re-entry path, route to SDeltaPhi-49.